Software Test Plan

Version: 2.0 Status: Updated Draft

Revision History

Version	Date	Change Summary	Status
1.0	Earlier version	Initial test-case list.	Superseded
2.0	11-11-2026	Expanded test planning sections, scope, strategy, environment, schedule, risks, metrics, traceability and test cases.	Updated draft

1. Introduction

1.1 Purpose

This Software Test Plan defines the scope, approach, environment, schedule, deliverables, responsibilities, risks and completion criteria for testing the Hostel Management System.

1.2 Scope

Testing covers student and warden authentication, student registration/profile, room information, fees and payment workflow, facility booking, leave, mess, laundry, complaints, lost and found, notifications and role-based access. Testing includes automated functional and route/database interaction tests using pytest and Flask Test Client, with additional manual and non-functional tests planned where needed.

1.3 References

- Hostel Management System SRS, updated version 2.0.
- Hostel Management System SAD, updated version 2.0.
- Existing test case documents and pytest test suite.
- Software Engineering Unit 4 lecture material on software quality metrics, measurement and testing.

1.4 Definitions

Term	Meaning
STP	Software Test Plan
UT	Unit / focused test case identifier used in the earlier test-case list
ST	System test scenario identifier
RTM	Requirements Traceability Matrix
UAT	User Acceptance Testing
SUT	System Under Test

2. Test Items

- Student registration and authentication.
- Student dashboard, profile and protected records.

- Warden authentication, dashboard and role access.
- Room details and room availability.
- Fee records, payment workflow, payment history and receipt.
- Common facility booking.
- Leave application and status.
- Mess menu, meal registration, attendance, feedback and related records.
- Complaint submission, attachment and status.
- Lost and found, laundry and notifications where enabled.

3. Features to be Tested

Feature	SRS Requirement IDs	Testing Focus
Registration and authentication	FR-001, FR-002, SR-001, SR-003	Valid/invalid input, session behaviour, password handling.
Profile and student data access	FR-003, SR-006	Profile retrieval, ownership and privacy.
Room management	FR-004	Availability, hostel filtering, protected access and consistency.
Facility booking	FR-005	Available slot, booking creation, conflicts and history.
Fees and payment	FR-006, FR-013	Fee visibility, payment workflow, history, receipt and ownership.
Mess management	FR-007, FR-014	Menu display, meal actions, warden CRUD and duplicate validation.
Leave, laundry, complaints, lost/found	FR-008 to FR-011	Submission, validation, status, ownership and attachment handling.
Warden access	FR-012, SR-002	Role restrictions and administrative operations.
Security and data integrity	SR-001 to SR-006, NFR-003, NFR-008	Unauthorized access, invalid input and record consistency.

4. Features Not to be Tested

- Internal operation of third-party payment provider or banking systems.
- Internal infrastructure of email/SMS providers.
- Physical hostel operations such as actual room inspections, meal preparation or laundry collection.
- Production-scale load limits unless a separate performance environment and workload are provided.
- Penetration testing or formal compliance certification; these require a dedicated security assessment.

5. Test Approach / Strategy

5.1 Test Levels

- Unit/focused testing: verify individual validation, helper and business-rule behaviour where isolated tests exist.
- Integration testing: verify Flask routes, authentication, SQLAlchemy models and database interactions together.
- System testing: verify complete user workflows and role-specific access across modules.

- Acceptance testing: confirm representative student and warden tasks against the SRS acceptance criteria.

5.2 Test Types

- Functional testing: verify each feature against its expected behaviour.
- Negative testing: submit missing, malformed, duplicate or unauthorized requests.
- Regression testing: rerun the automated suite after code or template changes.
- Security testing: verify authentication, role authorization, ownership checks and safe input handling.
- Usability testing: inspect navigation, labels, validation messages and responsive presentation.
- Performance testing: measure response time and concurrency only in a controlled environment.

5.3 Entry Criteria

- A stable build is available and dependencies are installed.
- Database configuration and test database are available.
- Test accounts and representative test data are prepared.
- Test cases and expected results are reviewed against the SRS.
- Application starts successfully in the test environment.

5.4 Exit Criteria

- All planned high-priority test cases have been executed or explicitly marked blocked/not applicable.
- No unresolved critical defect remains.
- Automated regression suite completes successfully.
- Failed tests are documented with reproducible steps and defect references.
- Requirement coverage and residual risks are reported.

5.5 Security Validation

- Verify login required for protected pages.
- Verify students cannot access warden-only operations.
- Verify students cannot view or pay another student’s fees.
- Verify invalid form data is rejected server-side.
- Review password hashing, secret configuration, upload validation and HTTPS deployment settings.

5.6 Test Design Techniques

- Positive testing: provide valid inputs and verify successful completion of the intended operation.
- Negative testing: provide missing, malformed, duplicate or unauthorized inputs and verify safe rejection.
- Boundary testing: test values at, below and above defined field limits, such as contact-number length, required fields and upload restrictions.
- Role/ownership testing: repeat relevant operations as student, warden and unauthenticated user to verify access boundaries.
- Regression testing: rerun the automated suite after application, database, route or template changes.

5.7 Verification and Validation

Verification checks whether the implementation and work products conform to the specified requirements and design. Validation checks whether the resulting system supports the users' intended hostel-management tasks. The plan uses both: requirement/design reviews and automated tests for verification, and representative student/warden workflow walkthroughs for validation.

Testing objectives include demonstration of acceptable operation, detection of defects and limitations, and prevention of future defects through feedback to requirements, design and implementation.

6. Test Environment

Category	Environment / Details
Hardware	Developer workstation or compatible server; client browser on desktop/laptop. Mobile browser checks are

	planned.
Software	Python runtime, Flask application, SQLAlchemy, configured relational database; SQLite used by automated tests.
Test Framework	pytest and Flask Test Client.
Version Control / CI	Git and GitHub; GitHub Actions used for automated test execution where configured.
Test Data	Dedicated test students, warden account, room records, fee records, bookings, leave applications, mess menu data and complaint records. Use synthetic data only.
Isolation	Automated tests use a separate test database to avoid modifying production/development records.

6.1 Code Coverage Measurement

Code coverage measures the fraction of the code under test reached by the existing tests. Structural coverage measures include statement coverage, decision coverage and condition coverage. Statement coverage checks whether executable statements were run; decision coverage checks whether decision outcomes (such as true and false branches) were exercised; condition coverage checks whether individual Boolean conditions have been evaluated to both true and false where applicable.

A combined coverage value must not be presented as a separate branch-coverage value unless the coverage tool reports branch coverage explicitly. Passing tests alone do not establish 100% code or branch coverage.

The Unit 4 material discusses aiming for approximately 80% coverage as a practical target while recognizing diminishing returns; this is a guideline, not a guarantee that uncovered code is risk-free. Critical security, payment, authorization and data-integrity paths may require targeted additional tests even when overall coverage is above the target.

7. Test Schedule

The project does not provide an approved calendar schedule in the supplied documents. The following phase-based schedule is used until the team confirms exact dates.

Phase	Activity	Planned Timing	Completion Evidence
1	Review SRS/SAD and finalize test cases.	Before execution	Reviewed test cases and RTM.
2	Prepare test environment, accounts and test data.	After test design	Environment checklist.
3	Execute automated unit/focused and integration tests.	After environment setup	pytest output and CI result.
4	Execute system and negative tests.	After automated baseline	System test log and defect list.
5	Perform usability/security/performance checks where environment permits.	Before acceptance	Review notes and measurement results.
6	Prepare test summary and sign-off.	After exit criteria	Final test summary and approvals.

7.1 Recorded Automated Test Execution Snapshot

The recorded automated test summary supplied for this project reports 223 tests executed, 223 passed, 0 failed, an execution time of 48.62 seconds and 81% combined coverage. The combined coverage figure is retained as reported; statement and branch percentages were not separately supplied and are therefore not inferred.

Metric	Recorded Result	Interpretation
Total automated tests	223	Reported test-suite execution count.
Passed	223	All reported automated tests passed.
Failed	0	No failing tests in the recorded run.
Execution time	48.62 seconds	Recorded test execution duration.
Combined coverage	81%	Reported combined figure; not a separately verified branch-coverage percentage.
Warnings	7	Warnings were reported and did not cause test failures.

7.2 Coverage Gaps and Follow-up

- Generate a detailed coverage report that lists statement and branch coverage separately for the application package.
- Review uncovered lines and branches, prioritizing authorization, validation, payment, file-upload and database error paths.
- Add tests for unverified or planned areas such as laundry, lost-and-found, notifications, browser compatibility, performance and external provider integration where those features are enabled.
- Retain the coverage HTML/XML report and CI execution log as evidence for the final submission.

8. Test Deliverables

- Software Test Plan (this document).
- Test case catalogue and requirement traceability matrix.
- Automated test scripts and test data.
- Test execution logs and CI results.
- Defect reports with severity, reproduction steps and status.
- Coverage and quality metric report when measured.
- Final Test Summary Report and approval record.

9. Roles and Responsibilities

Role	Responsibilities
Test Lead	Maintain the test plan, coordinate execution, review results and report test status.
Test Engineer	Design and execute test cases, maintain test data, record results and report defects.
Developer	Support test environment, investigate defects, implement fixes and support regression testing.
Project Guide / Evaluator	Review deliverables, provide feedback and approve

	acceptance where required.

10. Risks and Mitigation

Risk	Impact	Mitigation
Test database unavailable	Tests cannot execute reliably.	Maintain separate test configuration and verify database setup before execution.
Test data is incomplete	Some workflows cannot be validated.	Prepare synthetic accounts and related records for each module.
External payment/messaging dependency unavailable	End-to-end external integration cannot be verified.	Use sandbox/stub for functional tests and clearly report integration limitation.
UI changes invalidate old assertions	CI failures or misleading results.	Update tests only after confirming intended UI behaviour; preserve application functionality.
Limited manual testing time	Some acceptance criteria remain unverified.	Prioritize high-risk workflows and record unexecuted cases explicitly.
Unmeasured performance/security claims	Quality status may be overstated.	Do not report numeric targets or certification without controlled measurement.

11. Assumptions and Dependencies

- A stable application build and test database are available.
- Test users and warden credentials can be provisioned safely.
- The automated suite can run with the project dependencies installed.
- Payment gateway and messaging services may be simulated or unavailable in the test environment.
- The final feature set and route permissions are defined by the current SRS and implementation.

11.1 Test Limitations and Acceptance Note

The recorded result is evidence of the automated pytest run and its reported combined coverage. It does not establish that every test case in this plan was executed manually, that all browser/device combinations were checked, or that production payment, messaging, load, penetration or accessibility testing was completed. Test cases marked planned, partial or not verified must remain in those states until supported by execution evidence.

12. Suspension and Resumption Criteria

12.1 Suspend Testing If

- The application cannot start or the test database is unavailable.
- A blocking defect prevents execution of a substantial portion of planned cases.
- The build changes during execution and invalidates the test baseline.
- Test data corruption makes results unreliable.

12.2 Resume Testing When

- The blocking defect is fixed or a stable build is restored.
- The test database and test data are verified.
- The affected test cases are reviewed and the test baseline is recorded.

13. Test Case Management and Traceability

Test cases use stable identifiers and map to SRS requirement IDs. Each execution record should include build/commit, date, tester, preconditions, test data, steps, expected result, actual result, status and defect reference. The RTM shall be updated when a requirement or test case changes.

SRS Requirement	Test Case IDs
FR-001	TC-REG-01, TC-REG-02
FR-002, SR-001, SR-003	TC-AUTH-01 to TC-AUTH-04
FR-003, SR-006	TC-PROF-01, TC-SEC-01
FR-004	TC-ROOM-01 to TC-ROOM-04
FR-005	TC-FAC-01 to TC-FAC-03
FR-006, FR-013	TC-FEE-01 to TC-FEE-04, TC-WFEE-01
FR-007, FR-014	TC-MESS-01 to TC-MESS-04, TC-WMENU-01 to TC-WMENU-03
FR-008	TC-LEAVE-01 to TC-LEAVE-03
FR-009	TC-LAUN-01
FR-010	TC-LF-01
FR-011	TC-COMP-01 to TC-COMP-03
FR-012, SR-002	TC-WARD-01 to TC-WARD-03

14. Test Metrics and Reporting

Metric	Definition	Reporting Method
Test execution percentage	Executed test cases / planned test cases × 100.	Test execution log.
Pass percentage	Passed test cases / executed test cases × 100.	Test execution log.
Fail percentage	Failed test cases / executed test cases × 100.	Test execution log.
Requirement coverage	Requirements with at least one mapped test / total testable requirements × 100.	RTM.
Defect density	Number of confirmed defects relative to a defined size measure.	Defect log and agreed size measure.
Defect age	Time from defect reporting to closure.	Defect tracker.
Code/branch coverage	Executed statements/branches relative to measurable statements/branches.	Coverage tool output; not inferred from pass count.

Known automated-suite baseline from the latest project work: 223 passed, 0 failed and 7 warnings. This is a pytest suite execution summary, not a measured code/branch coverage percentage and not evidence that every planned manual or external-integration test has been executed.

15. Test Cases

The table below combines the five existing test cases with additional high-value scenarios from the SRS and current automated test scope. “Existing automated evidence” is used only where supported by the inspected test-suite results; other cases remain planned until separately executed.

Test Case ID	Module	Test Objective / Expected Result	Actual Result / Evidence	Status
UT-01	Student Registration	Valid student registration creates an account.	Not separately verified in recorded coverage summary.	NOT VERIFIED
UT-02	Common Facility Booking	Valid available slot can be booked and appears in booking history.	Automated tests passed for common facility booking functionality.	PASS
UT-03	Fee Management	Payment workflow updates payment record/history and receipt information.	Automated tests passed for fee payment processing and payment history functionality.	PASS
UT-04	Leave Management	Valid leave request is stored and status/history is available.	Automated tests passed for leave management functionality.	PASS
UT-05	Complaint Management	Complaint submission and related operations work correctly.	Automated tests passed for complaint submission and related functionality.	PASS
TC-AUTH-01	Student Authentication	Valid student credentials authenticate and open student dashboard.	Automated route tests cover successful login.	Automated suite evidence
TC-AUTH-02	Invalid Authentication	Invalid credentials are rejected with an error.	Automated route tests cover failed login.	Automated suite evidence
TC-AUTH-03	Logout	Logout ends the authenticated session.	Automated route tests cover logout.	Automated suite evidence
TC-AUTH-04	Protected Pages	Unauthenticated users cannot access protected pages.	Automated route/security tests cover protected access.	Automated suite evidence
TC-ROOM-01	Room Availability	Room availability information is displayed correctly.	Automated room tests cover availability.	Automated suite evidence
TC-ROOM-02	Hostel Filtering	Room results respect hostel selection/filtering.	Automated room tests cover hostel filtering.	Automated suite evidence
TC-ROOM-03	Unsupported Allocation	System does not permit unsupported room allocation operation.	Automated room tests cover allocation limitation.	Automated suite evidence

TC-FEE-01	Own Fee Access	Student can view their own fee information.	Automated fee tests cover own-fee access.	Automated suite evidence
TC-FEE-02	Cross-Student Fee Access	Student cannot access another student's fee record.	Automated tests cover access restriction.	Automated suite evidence
TC-FEE-03	Warden Fee Creation	Authorized warden can create fee records.	Automated warden tests cover fee creation.	Automated suite evidence
TC-WARD-01	Warden Login	Valid warden credentials open warden dashboard.	Automated warden authentication tests pass.	Automated suite evidence
TC-WARD-02	Warden Access Control	Student cannot access warden-only operations.	Automated warden access tests pass.	Automated suite evidence
TC-WMENU-01	Mess Menu Create	Warden can create a valid menu entry.	Automated warden mess-menu tests pass.	Automated suite evidence
TC-WMENU-02	Mess Menu Update/Delete	Warden can update or delete a menu entry.	Automated warden mess-menu tests pass.	Automated suite evidence
TC-WMENU-03	Duplicate Menu Validation	Duplicate or invalid menu entries are rejected.	Automated warden mess-menu tests cover duplicate validation.	Automated suite evidence
TC-LEAVE-01	Leave Validation	Missing or invalid leave data is rejected.	Further individual execution record required.	PLANNED
TC-COMP-01	Complaint Attachment	Attachment upload and ownership restrictions are enforced.	Automated complaint tests cover attachment-related behaviour.	Automated suite evidence
TC-LAUN-01	Laundry Request	Student submits a laundry request and views status.	Separate execution evidence required.	PLANNED
TC-LF-01	Lost and Found	Student reports/searches/claims an item.	Separate execution evidence required.	PLANNED
TC-NOT-01	Notifications	Relevant notification is displayed to intended user.	Separate execution evidence required.	PLANNED
TC-PERF-01	Performance	Measure representative page and transaction response time.	No performance measurement supplied.	PLANNED
TC-SEC-01	Data Ownership	Student cannot access another student's private record.	Automated fee and complaint tests provide partial evidence; broader coverage required.	PARTIAL

TC-SEC-02	Role Authorization	Warden-only actions reject student sessions.	Automated warden access tests pass.	Automated suite evidence

16. Reporting and Sign-Off

A final test summary shall report executed, passed, failed, blocked and not-run cases; defects by severity; requirement coverage; known limitations; and recommendations. Sign-off is granted only after the exit criteria are reviewed by the responsible project stakeholders.